

This implementation is a **full-stack Training Management System (TMS)** designed to handle the academic workflow between students, instructors, and administrators. Let me break it down clearly:

High-Level Purpose

The system is built to:

- Allow **students** to browse courses, enroll, track GPA/credits, and view transcripts.
- Enable **instructors** to approve enrollments in real time and submit grades with automated letter conversion.
- Give **administrators** control over course catalogs, capacity limits, and rollback safeguards when conflicts occur.
- Provide **real-time synchronization** using **SignalR WebSockets**, so changes instantly appear across all connected clients without page refresh.

Feature Implementation by Persona

- **Student Portal:** Interactive course browser, one-click enrollment, GPA tracker, transcript ledger.
- **Instructor Portal:** Approval queue, distribution charts, grade submission with RxJS `exhaustMap` to prevent double-submits.
- **Admin Control:** Course CRUD, optimistic deletion with rollback if active enrollments exist.

System Workflows

The implementation is illustrated through **sequence diagrams**:

1. Authentication & Token Rotation

- Short-lived JWT access tokens + long-lived refresh tokens.
- Theft detection: if a refresh token is reused, all sessions are revoked.

2. Student Enrollment & Instructor Approval

- Student submits enrollment → API validates capacity → status set to *Pending*.

- Instructor approves → SignalR broadcasts update → student sees *Approved* instantly.

3. Grade Submission & Transcript Update

- Instructor enters score via slider → mapped to letter grade → saved in DB.
- SignalR pushes update → student's GPA recalculates live.

4. Admin Course Deletion & Rollback

- Optimistic UI removal → server checks enrollments.
- If conflict (active students), rollback restores course row with error toast.

Architectural Highlights

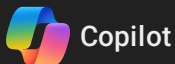
- NgRx SignalStore: Entity adapters, computed state, optimistic updates, rollback on API conflicts.
- SignalR Real-Time Sync: Full-duplex WebSocket communication with automatic reconnection.
- Clean Architecture + CQRS: Thin controllers delegate to domain services, MediatR handlers, validation pipeline, RFC 7807 error handling.

In short, this implementation is a **robust enterprise-grade academic platform** that combines **Angular frontend state management**, **ASP.NET Core backend with CQRS**, and **SignalR real-time synchronization** to ensure smooth, immediate, and reliable academic workflows across all roles.

Would you like me to dive deeper into the authentication flow, the real-time enrollment approval, or the grade submission pipeline?

You 

give me a sequence diagram but in words what the app can perform.



Got it — you want a **sequence diagram in words** that describes what the app can perform. Instead of the technical mermaid syntax, I'll narrate the flow step-by-step like a verbal diagram. Here's the breakdown:

Sequence Diagram in Words — App Capabilities

1. Login & Persona Switch

- A user opens the app and either logs in with credentials or uses the demo persona switcher.
 - The system authenticates, issues tokens, and directs the user to the correct dashboard (Student, Instructor, or Admin).
 - A welcome toast confirms successful entry.
-

2. Student Enrollment Lifecycle

- A student browses the course catalog, sees seat availability, and clicks “Enroll.”
 - The system validates capacity, saves the enrollment as *Pending*, and pushes a real-time update to both student and instructor dashboards.
 - The instructor sees the request in their queue and approves or rejects.
 - The student’s transcript ledger updates instantly with the new status.
-

3. Instructor Grade Submission

- An instructor selects a student and course, enters a numerical score via slider.
 - The system maps the score to a letter grade (e.g., 92.5% → A+).
 - On publish, the grade is saved in the database and broadcast via SignalR.
 - The student’s GPA recalculates live and the transcript updates with the new grade.
-

4. Admin Course Management

- An admin creates, edits, or deletes courses.
 - On deletion, the UI optimistically removes the course row.
 - The backend checks for active enrollments.
 - If found, the deletion is rejected, and the UI rolls back instantly with an error toast.
 - If no conflict, the deletion is finalized.
-

5. Cross-Cutting Real-Time Sync

- Any action (enrollment, approval, grade, deletion) triggers a SignalR broadcast.
 - All connected browsers update without refresh.
 - A connection badge shows live status (Connected, Reconnecting, Offline).
 - Toast notifications alert users about approvals, grades, or errors.
-

👉 In words, the app performs a **continuous loop of user actions → backend validation → database update → SignalR broadcast → instant UI reflection** across all personas.

Would you like me to expand this into a full narrative flow diagram that strings all personas together in one end-to-end story (login → enrollment → approval → grading → admin rollback)?